CSCI 6461 | TOPIC 4

AArch64 Assembly Programming: Arithmetic, Logic, & Control Flow

ALU mechanics, shifted-register operands, bitwise operations, condition flags, structured branches, and conditional selection

Dr. J. Burns
Computer Systems Architecture — AArch64 Reference Platform

Topic 4: Core Lecture Outline

1. **ALU Operations: Arithmetic, Shifting, & Bitwise Logic**
   - Addition, subtraction, multiplication, division, and shifted-register operands.
   - Bitwise logic (and, `orr`, `eor`, `bic`) and bitfield extraction (`ubfx`).
2. **Condition Flags & Comparison Mechanics**
   - NZCV flag generation, comparison instructions (`cmp`, `tst`), and signed vs. unsigned branches.
3. **Control Flow: Branches & Branchless Selection**
   - Translating loops and `if-else` structures; branch-and-test (`cbz`/`cbnz`).
   - Branch penalties and branchless conditional selection (`csel`, `cset`).

PART 1

ALU Operations: Arithmetic, Shifting, & Bitwise Logic

Arithmetic Primitives: Addition & Subtraction

- Many AArch64 data-processing instructions use a **3-operand non-destructive format**. Flag-setting forms such as `adds` and `subs` update NZCV; `add` and `sub` do not.

Instruction	Operation	Flags Updated?
`add x0, x1, x2`	$x_0 \leftarrow x_1 + x_2$	No
`adds x0, x1, x2`	$x_0 \leftarrow x_1 + x_2$	**Yes (NZCV)**
`sub x0, x1, x2`	$x_0 \leftarrow x_1 - x_2$	No
`subs x0, x1, x2`	$x_0 \leftarrow x_1 - x_2$	**Yes (NZCV)**
`neg x0, x1`	$x_0 \leftarrow 0 - x_1$	No; alias of `sub x0, xzr, x1`

Immediate Constant Arithmetic

`add x0, x1, #42` uses a 12-bit immediate. The add/sub immediate form can also shift that field left by 12 bits. The corresponding w-register form performs 32-bit arithmetic.

Shifted-Register Operands and Shift Operations

- Many AArch64 data-processing instructions can encode a shifted register operand directly in the instruction.
- Execution latency for shifted operands is implementation-dependent.

Shift Type	Mnemonic	Operation Meaning
Logical Shift Left	`lsl #k`	Multiply by 2^k modulo the register width; zeros enter the low bits.
Logical Shift Right	`lsr #k`	Unsigned floor division by 2^k; zeros enter the high bits.
Arithmetic Shift Right	`asr #k`	Sign-preserving right shift; the sign bit is replicated into the high bits.

Integrated Shift-Add Examples

`add x0, x1, x2, lsl #3` $\quad x_0 \leftarrow x_1 + (x_2 \times 8)$

`sub x0, x1, x2, lsr #1` $\quad x_0 \leftarrow x_1 - \lfloor x_2/2 \rfloor$ for unsigned x_2

Integer Multiplication & Division

- AArch64 defines instructions for integer multiplication and division, leaving the underlying microarchitectural execution units implementation-dependent.

Instruction	RTL Semantic Action	Description
`mul x0, x1, x2`	$x_0 \leftarrow x_1 \times x_2$	Lower 64 bits of the product
`madd x0, x1, x2, x3`	$x_0 \leftarrow x_3 + (x_1 \times x_2)$	Multiply-Add
`msub x0, x1, x2, x3`	$x_0 \leftarrow x_3 - (x_1 \times x_2)$	Multiply-Subtract
`sdiv x0, x1, x2`	$x_0 \leftarrow x_1 / x_2$	Signed division
`udiv x0, x1, x2`	$x_0 \leftarrow x_1 / x_2$	Unsigned division

Division by Zero

AArch64 integer division by zero does **not** raise an exception; it returns `0` in the destination register. For signed division, the overflow case $\mathrm{INT64\_MIN}/(-1)$ also does not trap; the destination receives the architecturally defined 64-bit result.

Bitwise Logic Primitives

- Bitwise operations apply independently to each corresponding bit position.

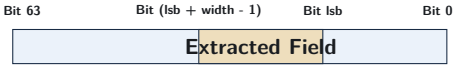
Instruction	Operation	Bitwise RTL Formula
`and x0, x1, x2`	Bitwise AND	$x_0 \leftarrow x_1 \wedge x_2$
`orr x0, x1, x2`	Bitwise OR	$x_0 \leftarrow x_1 \vee x_2$
`eor x0, x1, x2`	Bitwise XOR	$x_0 \leftarrow x_1 \oplus x_2$
`bic x0, x1, x2`	Bit Clear (AND NOT)	$x_0 \leftarrow x_1 \wedge (\sim x_2)$
`mvn x0, x1`	Bitwise NOT	$x_0 \leftarrow \sim x_1$ (Alias: `orn x0, xzr, x1`)

Bit Clear (`bic`) Idiom

`bic x0, x1, x2` sets bits to `0` in x_1 wherever the mask in x_2 has `1`s.

Bitfield Extraction: `ubfx`

- AArch64 provides bitfield instructions that extract packed fields without explicit mask-and-shift sequences.
- `ubfx xd, xn, #lsb, #width`: unsigned bitfield extract.



Example: Extract 8-bit Field Starting at Bit 16

Mask + shift: `lsr x0, x1, #16` → `and x0, x0, #0xff`
Bitfield extract: `ubfx x0, x1, #16, #8`

PART 2

Condition Flags & Comparison Mechanics

The NZCV Condition Flags

- Flag bits reside in `pstate` and record status of the most recent instruction that updates NZCV.
- Common flag-setting forms include instructions with an "s" suffix and explicit comparisons.

Flag	Name	Condition Set to 1
N	**Negative**	The most-significant bit of the result is 1.
Z	**Zero**	Result is exactly zero.
C	**Carry**	Unsigned carry-out (addition) or **no borrow** (subtraction).
V	**Overflow**	The signed mathematical result cannot be represented in the destination width.

Carry Flag in Subtraction

In AArch64 subtraction ($A - B$), C $= 1$ indicates $A \geq B$ (no borrow required). C $= 0$ indicates $A < B$ (unsigned borrow occurred).

Comparison Instructions: `cmp` and `tst`

- Comparison aliases set NZCV flags and discard the computed result.

Instruction	Equivalent Operation	Flags Set
`cmp x0, x1`	`subs xzr, x0, x1`	NZCV from $x_0 - x_1$
`cmn x0, x1`	`adds xzr, x0, x1`	NZCV from $x_0 + x_1$
`tst x0, x1`	`ands xzr, x0, x1`	N/Z from the AND result; C/V cleared

Checking Flags After `cmp x0, x1`

- $x_0 == x_1$: $Z = 1$.
- $x_0 < x_1$ (signed): $N \neq V$.
- $x_0 < x_1$ (unsigned): $C = 0$.

Condition Codes: Signed vs. Unsigned Branches

Branch	Meaning	Flag Condition	Comparison Context
`b.eq`	Equal	$Z == 1$	Signed / Unsigned
`b.ne`	Not Equal	$Z == 0$	Signed / Unsigned
`b.lt`	Signed Less Than ($<$)	$N \neq V$	**Signed**
`b.le`	Signed Less or Equal ($\leq$)	$(Z == 1) \vee (N \neq V)$	**Signed**
`b.gt`	Signed Greater Than ($>$)	$(Z == 0) \wedge (N == V)$	**Signed**
`b.ge`	Signed Greater or Equal ($\geq$)	$N == V$	**Signed**
`b.lo`	Unsigned Lower ($<$)	$C == 0$	**Unsigned**
`b.hs`	Unsigned Higher or Same ($\geq$)	$C == 1$	**Unsigned**
`b.hi`	Unsigned Higher ($>$)	$(C == 1) \wedge (Z == 0)$	**Unsigned**
`b.ls`	Unsigned Lower or Equal ($\leq$)	$(C == 0) \vee (Z == 1)$	**Unsigned**

Usage Context

Unsigned conditions are appropriate when the program semantics treat the values as unsigned; they are also commonly used for address and range comparisons.

PART 3

Control Flow: Branches & Selection

Compare-and-Branch Instructions: cbz and cbnz

- **cbz / cbnz** test a register directly and do not alter NZCV.
- They encode the zero test and branch in one architectural instruction instead of a separate compare and conditional branch.

Null Pointer Validation

```
cbz x0, .Lnull   // branch if null
ldr x1, [x0]     // dereference
```

Loop Counter Decrement

```
.Lloop:
sub x1, x1, #1    // count–
cbnz x1, .Lloop   // repeat if nonzero
```

Single-Bit Test: `tbz x0, #0, label` branches if bit 0 is zero.

Translating `if-then-else` Blocks

High-Level C Code

```
if (x > y) {
    result = x - y;
} else {
    result = y - x;
}
```

AArch64 Assembly Mapping

```
// Assume: x, y are signed int64_t
// x0=x, x1=y, x2=result
cmp x0, x1        // Compare x and y
b.le .Lelse       // Invert test: jump if x <= y
sub x2, x0, x1    // Then-body: result = x - y
b .Lend           // Skip else block
.Lelse:
sub x2, x1, x0    // Else-body: result = y - x
.Lend:
```

Assembly Pattern: Invert the high-level condition to fall through into the "then" path.

Loop Translation: Inverted Loop Pattern

High-Level C Loop

```
// Compute array sum
int64_t i = 0;
int64_t n = ...; // signed bound
while (i < n) {
    sum += arr[i];
    i++;
}
```

Bottom-Tested AArch64 Loop

```
// x0=arr, x1=n, x2=i, x3=sum
mov x2, xzr                 // i = 0
mov x3, xzr                 // sum = 0
b .Ltest                    // Jump to test first
.Lbody:
ldr x4, [x0, x2, lsl #3]    // Load arr[i]
add x3, x3, x4              // sum += arr[i]
add x2, x2, #1              // i++
.Ltest:
cmp x2, x1                  // i < n?
b.lt .Lbody                 // Loop if true
```

Common Loop Layout: Placing the test at the bottom gives the steady-state loop body one loop-closing conditional branch.

Branch Misprediction & Conditional Selection (`csel`)

- A mispredicted conditional branch discards younger speculative work; the cost is implementation-dependent.
- **`csel`** chooses between two register operands from NZCV without changing control flow.
- **Trade-off:** Conditional selection avoids prediction for that decision but can extend a data-dependency chain.

Signed Maximum: `result` $= (x_1 > x_2)\ ?\ x_1 : x_2$

```
cmp x1, x2              // set NZCV from x1 - x2
csel x0, x1, x2, gt     // signed max; use hi for unsigned
```

Related Forms

`cset w0, eq` sets 0 or 1 from a condition; `cinc x0, x1, eq` conditionally increments `x1`.

Review and Self-Test Questions

Topic 4: Comprehensive Summary

- **Arithmetic & Shifts:** Regular register and immediate arithmetic includes shifted-register operands in many data-processing instructions.
- **Bitwise Operations:** and, `orr`, `eor`, `bic`, and bitfield instructions operate directly on packed binary data.
- **NZCV Flags:** Flag-setting arithmetic and aliases such as `cmp` and `tst` support signed and unsigned conditions.
- **Control Flow:** Compilers commonly arrange one path as fall-through and use bottom-tested loops where appropriate.
- **Conditional Selection:** `csel` and related forms can replace short conditional branches when data-dependent selection is preferable.

Self-Test Questions: Arithmetic & Logic

1. **Shifted Addition:** What is the exact mathematical operation and resulting decimal value stored in `x0` after executing:

```
mov x1, #12  mov x2, #5  add x0, x1, x2, lsl #2
```

2. **Bit Clear Operation:** If register `x1` contains `0xff` and `x2` contains `0x0f`, what value resides in `x0` after executing `bic x0, x1, x2`?

3. **Signed vs. Unsigned Comparisons:** If `x0` contains -1 (`0xffffffff_ffffffff`) and `x1` contains 1, which branch will be taken after `cmp x0, x1`: `b.lt` or `b.lo`? Why?

4. **Flag Updating:** Explain why `add x0, x1, x2` does not update the Zero (Z) flag even if the resulting sum is exactly zero.

Self-Test Questions: Control Flow & Selection

1. **Bottom-Tested Loop Layout:** Why do compilers often place the conditional loop branch at the bottom of the loop body rather than at the top?
2. **Branchless Translation:** Write an equivalent 3-instruction branchless AArch64 sequence using `cmp`, `neg`, and `csel` to implement:

```
int64_t abs_val = (val < 0) ?  -val :  val;
```

3. **Compare-and-Branch Instructions:** When should an assembly programmer use `cbz x0, label` instead of `cmp x0, #0` followed by `b.eq label`?
4. **Branch Misprediction Trade-off:** What factors determine whether `csel` or a conditional branch is faster?